

STATERA AI Customer Support & Product Troubleshooting

Automated Evaluation & Test Suite (POC Architecture & Implementation Specification)

A production-grade test automation and LLM/RAG evaluation framework tailored for STATERA (www.statera.org), the environmental sampling technology leader behind the Composite Integrative Passive Sampler (CIPS). This POC fulfills all competencies of the AI Test Engineer role: combining software test automation, DeepEval evaluation metrics, multi-turn diagnostic state verification, IoT telemetry and MQTT mocking, adversarial red-teaming, load testing, and Azure DevOps CI/CD pipeline integration.

Target Repo: <https://github.com/smkpt/POC-STATERA.git> • Target Architecture: CIPS v2.2 Telemetry & Hybrid RAG

IMPORTANT Git Remote Push & Sandbox Network Permissions

Target repository: <https://github.com/smkpt/POC-STATERA.git>. In the standard sandbox environment, outgoing HTTPS traffic to GitHub requires network bypass permissions (BypassSandbox: true). If credentials or a Personal Access Token (PAT) are needed for pushing to smkpt/POC-STATERA, we will provide seamless automated push instructions or execute the push command with your approval.

NOTE Evaluation Framework Compatibility & Dual Execution Modes

The project incorporates the DeepEval framework (FaithfulnessMetric, AnswerRelevancyMetric, ContextualRelevancyMetric, HallucinationMetric, GEval, and LLMTestCase). In addition, we provide a self-contained, zero-dependency DeepEval evaluation adapter. This ensures the suite runs both in standard offline testing environments (no external API keys required) and in live CI environments with OpenAI/Azure OpenAI/Gemini keys.

01 Domain Architecture & Requirements Alignment

1. The STATERA Domain & Environmental Context

- **Product Focus:** Composite Integrative Passive Sampler (CIPS) and remote environmental monitoring telemetry units.
- **Analyte Scope:** Quantitative capture of 500+ organic contaminants in water, air, and sediment (PFAS, PCBs, PAHs, 6PPD, microcystins, pharmaceuticals, plasticizers) across log Kow < 0 to 8.3.
- **Field Realities:** Remote deployment sites (e.g., Tijuana River, Seattle, Cape Fear River, Kunming, Mekong River) subject to biofouling, battery depletion, solar charging fluctuations, LTE-M/NB-IoT/LoRaWAN/MQTT connectivity drops, and Performance Reference Compound (PRC) elution rate variations.
- **AI Agent Role:** Troubleshoot device failures, inspect telemetry feeds, guide field technicians through multi-turn diagnostic state trees, and handle warranty validations and automated RMA issuance.

02 Responsibilities Matrix & Implementation Mapping

Responsibility from Job Description	Implementation in POC
RAG retrieval accuracy, context relevance, semantic search precision, response quality	test_rag_deepeval.py + src/evaluation/metrics.py (measuring Context Recall, Precision, and RAG retrieval against CIPS documentation)
Measure faithfulness, answer relevance, hallucination rates, guardrails	DeepEval test cases (FaithfulnessMetric, AnswerRelevancyMetric, HallucinationMetric, SafetyGuardrailMetric)
Test multi-turn troubleshooting flows (connectivity, battery degradation, performance faults)	test_multiturn_troubleshooting.py validating state transitions across Cellular/MQTT drops, LiFePO4 battery degradation, and sampler biofouling
Simulate complex customer interactions for diagnostic state navigation	src/simulator/customer_simulator.py driving persona-based multi-turn dialogues (Field Technician, Lab Chemist, Frustrated Client)
Prevent regressions when product docs, firmware specs, or telemetry models change	test_regression_pipeline.py diffing semantic drift across CIPS Firmware v2.1 to v2.2 and updated sampling protocols
Automate API & E2E tests across AI agent, customer support, and warranty integrations	test_api_e2e_integration.py integrating mock REST/MQTT APIs for Device Telemetry and Warranty/RMA ERP endpoints
Run load and scalability tests on conversational APIs	test_load_scalability.py using asyncio to benchmark 10, 50, and 100 concurrent sessions, tracking RPS, P50, P95, and P99 latency
Adversarial tests covering prompt injection, out-of-scope requests, hallucination risks	test_adversarial_security.py red-teaming DAN exploits, system prompt extraction, fake telemetry payload attacks, and ungrounded model queries
Integrate AI evaluation and regression suites into Azure DevOps CI/CD	azure-pipelines.yml multi-stage pipeline with machine-enforced QA Go/No-Go gatekeeper

03 Proposed Project Structure

```

PROJECT DIRECTORY HIERARCHY - poc-statera

/Users/macuser/.gemini/antigravity/scratch/poc-statera/
├── README.md # Comprehensive project documentation & architecture guide
├── azure-pipelines.yml # Azure DevOps multi-stage CI/CD pipeline definition
├── .github/workflows/ai-eval-ci.yml # GitHub Actions CI/CD workflow
├── pyproject.toml # Python project configuration & metadata
├── pytest.ini # Pytest configuration with custom markers
├── requirements.txt # Dependency manifest (deepeval, pytest, asyncio, etc.)
├── docs/ # Design & Engineering Documentation
│   ├── ARCHITECTURE.md # AI Agent, RAG Pipeline & Telemetry architecture
│   ├── TESTING_STRATEGY.md # LLM test philosophy, DeepEval metrics, QA SLAs & Gate rules
│   ├── TROUBLESHOOTING_TAXONOMY.md # STATERA CIPS diagnostic state decision trees
│   └── AZURE_DEVOPS_INTEGRATION.md # Azure DevOps setup, test artifact publishing & quality gates
├── datasets/ # Golden Datasets & Fixtures
│   ├── golden_rag_eval.json # 25+ domain questions, retrieved contexts & ground truths
│   ├── multiturn_troubleshooting.json # Diagnostic multi-turn conversations (connectivity, battery, biofouling)
│   ├── adversarial_prompts.json # Prompt injections, jailbreaks, out-of-scope, and hallucination probes
│   └── telemetry_fixtures.json # Simulated device telemetry streams (MQTT topics, sensor telemetry)
├── src/ # Application & Evaluation Core
│   ├── agent/ # Conversational state machine, hybrid RAG & diagnostic tools
│   │   ├── conversational_agent.py # Multi-turn diagnostic agent with state management
│   │   ├── rag_engine.py # Semantic & hybrid search engine over CIPS documentation
│   │   └── tools.py # Telemetry query, Warranty lookup, RMA dispatch tools
│   ├── evaluation/ # Native DeepEval & mathematical fallback adapters
│   │   ├── deepeval_adapter.py # DeepEval wrapper & zero-dependency standalone fallback
│   │   ├── metrics.py # Faithfulness, Answer Relevancy, Context Recall & Precision
│   │   └── guardrails.py # PII masking, Prompt Injection, Out-of-Scope detectors
│   ├── mocks/ # Telemetry & Enterprise ERP Mocks
│   │   ├── telemetry_mock.py # Device telemetry & simulated MQTT broker
│   │   └── warranty_api_mock.py # REST API mock for warranty validation & RMA creation
│   └── simulator/ # Customer Simulator
│       └── customer_simulator.py # Persona-based multi-turn automated dialogue simulator
├── tests/ # Comprehensive Test Automation Suites
│   ├── conftest.py # Pytest fixtures, mock service lifecycle, test reports
│   ├── test_rag_deepeval.py # DeepEval RAG evaluations (faithfulness, relevancy, recall)
│   ├── test_multiturn_troubleshooting.py # Multi-turn diagnostic state navigation tests
│   ├── test_adversarial_security.py # Red-teaming, prompt injection, and guardrail enforcement
│   ├── test_api_e2e_integration.py # E2E API tests (Agent -> Telemetry Mock -> Warranty Mock)
│   ├── test_load_scalability.py # Asyncio load test measuring RPS, P50, P95, P99 latency
│   └── test_regression_pipeline.py # Firmware v2.1 vs v2.2 doc regression verification
├── scripts/ # Automation & CI/CD Scripts
│   ├── run_all_evals.py # Master test runner executing all evaluation stages
│   ├── generate_qa_report.py # Generates rich interactive HTML & Markdown QA reports
│   └── qa_gatekeeper.py # Enforces hard/soft thresholds and returns exit code 0 or 1
├── results/ # Evaluation Outputs & Artifacts
│   ├── evaluation_results.json # Machine-readable evaluation metrics
│   └── statera_qa_report.html # Rich HTML report with scorecards, breakdowns, and latency charts

```

04 Proposed Changes & File Details

Component 1: Design Documentation (docs/)

NEW docs/ARCHITECTURE.md

Detailed architectural breakdown of the AI agent orchestrator and integration interfaces.

- System components: Customer Interface, AI Agent Orchestrator, RAG Knowledge Base, Device Telemetry Hub, Warranty & ERP Gateway.
- Data flows for diagnostic troubleshooting: Telemetry pull -> RAG contextual lookup -> Diagnostic Tree decision -> Resolution / RMA dispatch.
- DeepEval testing framework integration architecture.

NEW docs/TESTING_STRATEGY.md

Complete QA Testing Methodology for GenAI & RAG systems.

- Metric definitions and mathematical formulation: Faithfulness, Answer Relevance, Context Recall, Context Precision, Hallucination Rate, Guardrail Compliance.
- Machine-enforced Go / No-Go Criteria Matrix (Hard blockers vs. Soft warnings).

NEW docs/TROUBLESHOOTING_TAXONOMY.md

Diagnostic state machines for Statera CIPS troubleshooting.

- Connectivity Failure: SIM/Cellular attachment, signal attenuation (RSSI < -105 dBm), MQTT timeout, APN mismatch.
- Battery Degradation: Low open-circuit voltage (< 3.0V), solar controller failure, cycle degradation (> 1500 cycles), temperature throttle.
- Performance Faults: Sampler biofouling, PRC elution anomalies, flow restriction, GCMS/LCMS extract volume errors.

NEW docs/AZURE_DEVOPS_INTEGRATION.md

Step-by-step setup guide for Azure DevOps CI/CD pipelines.

- Publishing JUnit test results, Cobertura coverage, and HTML evaluation artifacts.
- Gated deployment and branch protection policies based on QA Gatekeeper sign-off.

Component 2: Golden Datasets & Fixtures (datasets/)

NEW datasets/golden_rag_eval.json

25+ curated domain test cases based on www.statera.org technical literature.

- CIPS deployment duration (1-30 days), chemical extraction volume (4 mL solvent), LCMS/GCMS compatibility.
- log Kow range (< 0 to 8.3), compound classes (PFAS, PCBs, PAHs, 6PPD, microcystin LR, atrazine).
- Comparison against traditional LDPE and POCIS samplers; PRC calibration for water flow, temperature, and biofouling.

NEW datasets/multiturn_troubleshooting.json

Structured multi-turn conversation dialogues verifying diagnostic paths.

- Turn-by-turn inputs, expected agent diagnostic state, tool invocation requirements, and resolution criteria.

NEW datasets/adversarial_prompts.json

Test vectors for prompt injection, jailbreaks, and guardrail enforcement.

- Prompt injection, DAN jailbreaks, system prompt exfiltration, synthetic telemetry falsification, toxic queries, and hallucination traps (e.g., nonexistent CIPS-9900 nuclear sampler).

NEW datasets/telemetry_fixtures.json

Mock device telemetry messages and simulated MQTT broker payloads.

- Simulated streams across normal and faulty CIPS monitoring units (device IDs, timestamps, battery mV, solar current, RSSI, MQTT status, error flags).

Component 3: Core Implementation (src/)

NEW `src/agent/conversational_agent.py`

State-aware conversational agent handling customer troubleshooting inquiries.

- Tracks conversation state: INITIAL_QUERY -> DIAGNOSTIC_QUERY -> TELEMETRY_CHECK -> STEP_GUIDANCE -> RESOLVED / RMA_INITIATED.
- Integrates RAG retrieval and diagnostic tools.

NEW `src/agent/rag_engine.py`

Hybrid semantic & keyword retrieval engine indexed with Statera CIPS technical documentation, field guides, and calibration manuals.

NEW `src/agent/tools.py`

Tool definitions for device telemetry lookup, warranty entitlement verification, and RMA ticket generation.

NEW `src/evaluation/deepeval_adapter.py`

DeepEval framework adapter importing native deepeval when available, with a built-in standard-compliant fallback evaluating exact mathematical metrics.

NEW `src/evaluation/metrics.py`

Precise metric calculations for Faithfulness / Groundedness, Answer Relevance, Context Recall, Context Precision, and Hallucination Rate.

NEW `src/evaluation/guardrails.py`

Deterministic safety guards: PII redactor (regex + pattern), prompt injection detector, out-of-scope classifier, and toxicity filter.

NEW `src/mocks/telemetry_mock.py`

Mock IoT Telemetry API and simulated MQTT broker providing realistic device telemetry for CIPS units.

NEW `src/mocks/warranty_api_mock.py`

Mock REST/GraphQL API for warranty status checks, serial number validation, and RMA ticket creation.

NEW `src/simulator/customer_simulator.py`

Multi-persona customer simulator executing automated multi-turn diagnostic dialogues with the AI agent.

Component 4: Test Automation Suites (tests/)

NEW `tests/conftest.py`

Pytest fixtures, mock service setup/teardown, shared data loaders, and custom pytest hooks for evaluation reporting.

NEW `tests/test_rag_deepeval.py`

Evaluates RAG retrieval precision, context recall, faithfulness, and answer relevance on 25+ Statera golden cases using DeepEval test cases.

NEW `tests/test_multiturn_troubleshooting.py`

Tests multi-turn customer troubleshooting conversations across Cellular/MQTT disconnection, LiFePO4 battery degradation, and biofouling.

NEW `tests/test_adversarial_security.py`

Tests prompt injection attacks, jailbreak payloads, out-of-scope inquiries, and hallucination probes on fake Statera hardware.

NEW `tests/test_api_e2e_integration.py`

End-to-end integration tests: Agent queries Telemetry Mock -> evaluates diagnostic rule -> executes Warranty validation -> generates RMA ticket.

NEW `tests/test_load_scalability.py`

Asyncio load test simulating concurrent conversational sessions (10, 50, 100 concurrent users), verifying P50 < 800ms, P95 < 2500ms, and 0% error rate.

NEW

tests/test_regression_pipeline.py

Regression test suite comparing responses across CIPS Firmware v2.1 and v2.2 documentation changes, verifying no deprecated telemetry fields are cited.

Component 5: Automation Scripts, CI/CD, & Reports

NEW `scripts/run_all_evals.py`

Master CLI test runner executing the complete evaluation pipeline and producing results/evaluation_results.json.

NEW `scripts/generate_qa_report.py`

Generates interactive, standalone HTML test report (results/statera_qa_report.html) with metric scorecards, breakdowns, and latency charts.

NEW `scripts/qa_gatekeeper.py`

Automated gatekeeper script enforcing SLAs (Faithfulness $\geq 95\%$, Recall $\geq 90\%$, Zero PII/Jailbreaks) and returning exit code 0 (GO) or 1 (NO-GO).

NEW `azure-pipelines.yml`

Azure DevOps multi-stage YAML pipeline with stages: Lint_and_UnitTests, RAG_DeepEval_Suite, MultiTurn_and_E2E_APITests, Security_Adversarial_and_LoadTests, QA_Gatekeeper_and_SignOff.

NEW `.github/workflows/ai-eval-ci.yml`

GitHub Actions CI workflow executing the suite on pull requests and pushes to main branch.

NEW `README.md`

Comprehensive, executive-ready README detailing architecture, evaluation results, quickstart guide, test suites breakdown, and CI/CD setup.

05 Verification Plan & Execution Runbook

1. Full Test Suite Execution

```
$ cd /Users/macuser/.gemini/antigravity/scratch/poc-statera
$ python3 scripts/run_all_evals.py
```

2. QA Gatekeeper Sign-Off Verification (Machine-Enforced SLA)

```
$ python3 scripts/qa_gatekeeper.py --results results/evaluation_results.json
```

3. Pytest Discovery & Unit Verification

```
$ python3 -m unittest discover -s tests -p "test_*.py"
```

4. Interactive HTML QA Report Generation

```
$ python3 scripts/generate_qa_report.py --results results/evaluation_results.json \
$ --output results/statera_qa_report.html
```